

# **Desenvolvimento Web**

## **FoodDelivery – Paris**

**Autor:**

Érica Araujo de Jesus  
Mickael Cedraz Alencar  
Monyc Luisa Almeida de Cerqueira  
Nalbert de Souza Santana  
Pedro César Paixão de Jesus

Feira de Santana, 2025

# Agenda

Apresentar o desenvolvimento do sistema de pedidos de restaurante em Java utilizando Programação Orientada a objetos.

## **1. Problema:**

Apresentação do problema típico que o padrão de projeto resolve

## **2. Diagrama:**

Apresentação do Diagrama de Classe da Solução do Padrão de Projeto

## **3. Implementação:**

Explicação das etapas de implementação do padrão de projeto

## **4. Resultados:**

Diagrama de Classe e o código implementado para o padrão de projeto (antes e depois)

## **5. Considerações finais**

Apresentação das Considerações finais

# Problema

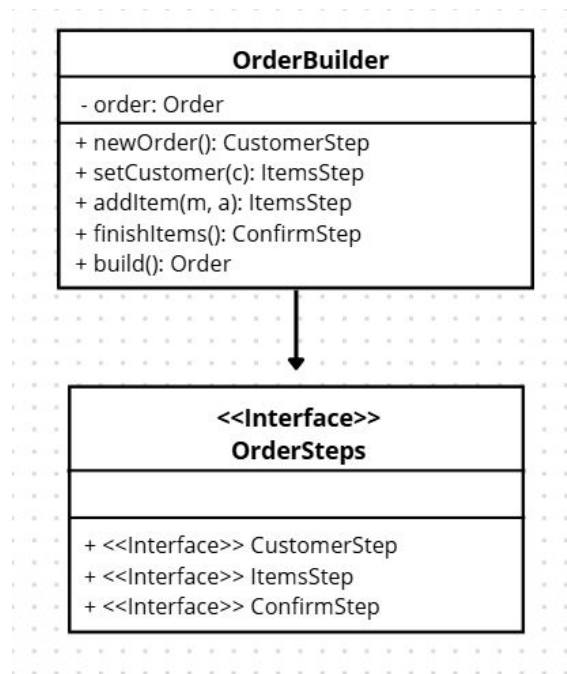


- A criação de pedidos é desorganizada e permite erros de sequência (ex: confirmar antes de adicionar itens).
- O padrão Builder organiza a construção do objeto em etapas bem definidas, por meio de interfaces específicas.
- Cada etapa leva apenas à próxima, garantindo a ordem correta e maior segurança na criação do pedido.
  - 
  -

# Diagrama de Classe

- **OrderBuilder** → Classe responsável por construir o objeto **Order** em etapas sequenciais.
- **OrderSteps** → Interface que define as etapas do processo (**CustomerStep**, **ItemsStep**, **ConfirmStep**), garantindo que o pedido seja montado na ordem correta.

A classe **OrderBuilder** implementa essas interfaces e contém o objeto **Order**, utilizando métodos como **newOrder()**, **setCustomer()**, **addItem()**, **finishItems()** e **build()** para encadear as etapas até gerar o pedido final.



# Implementação



Entregamos a classe OrderBuilder, seguindo o padrão Builder, para organizar a criação de pedidos.

O padrão garante que a sequência das etapas seja respeitada, prevenindo erros como pedidos sem itens ou quantidade acima do estoque.

O código foi testado e validado, e a criação de pedidos só é possível seguindo o fluxo correto.

As imagens a seguir mostram:

1. Erro ao criar pedido sem itens.
2. Erro ao adicionar item com quantidade maior que o estoque.
3. Pedido criado com sucesso, com cliente, itens e status corretos.

```
ID: 1
STATUS: ACCEPTED
CLIENTE: João
PEDIDOS
Item: Batata Frita | Quantidade: 2
Item: Hamburguer | Quantidade: 2
Pressione 0 para sair...
```

# Implementação



Entregamos a classe OrderBuilder, seguindo o padrão Builder, para organizar a criação de pedidos.

O padrão garante que a sequência das etapas seja respeitada, prevenindo erros como pedidos sem itens ou quantidade acima do estoque.

O código foi testado e validado, e a criação de pedidos só é possível seguindo o fluxo correto.

As imagens a seguir mostram:

1. Erro ao criar pedido sem itens.
2. Erro ao adicionar item com quantidade maior que o estoque.
3. Pedido criado com sucesso, com cliente, itens e status corretos.

```
ID: 1
STATUS: ACCEPTED
CLIENTE: João
PEDIDOS
Item: Batata Frita | Quantidade: 2
Item: Hamburguer | Quantidade: 2
Pressione 0 para sair...
```

Figura 3

# Implementação



```
Exception in thread "main" java.lang.IllegalStateException Create breakpoint : Pelo menos um item deve ser adicionado ao pedido
    at br.edu.unex.tiaLuDelivery.builders.OrderBuilder.finishItems(OrderBuilder.java:57)
    at br.edu.unex.tiaLuDelivery.cli.TiaLuCLI.start(TiaLuCLI.java:20)
    at br.edu.unex.tiaLuDelivery.TiaLuDeliveryApplication.main(TiaLuDeliveryApplication.java:7)

Process finished with exit code 1
```

Figura 1

```
Exception in thread "main" java.lang.IllegalStateException Create breakpoint : Pelo menos um item deve ser adicionado ao pedido
    at br.edu.unex.tiaLuDelivery.builders.OrderBuilder.finishItems(OrderBuilder.java:57)
    at br.edu.unex.tiaLuDelivery.cli.TiaLuCLI.start(TiaLuCLI.java:20)
    at br.edu.unex.tiaLuDelivery.TiaLuDeliveryApplication.main(TiaLuDeliveryApplication.java:7)

Process finished with exit code 1
```

Figura 2

# Considerações finais



A refatoração do sistema FoodDelivery utilizando o Builder Encadeado mostrou-se eficaz para organizar a criação de pedidos, garantindo a sequência correta, maior segurança e legibilidade do código. O padrão preveniu erros de inicialização e tornou o sistema mais claro e fácil de manter. Com mais tempo, seria possível aprimorar cada etapa antes de implementar funcionalidades adicionais, reforçando a confiabilidade do processo.



# Referências



**Sommerville, I.** Software Engineering, 9th edition, Addison-Wesley, England, 2011.

**DevMedia.** “Conceitos e exemplos: Polimorfismo na Programação Orientada a Objetos”, Disponível em <https://www.devmedia.com.br/conceitos-e-exemplos-polimorfismoprogramacao-orientada-a-objetos/18701>, Accessed October 2025.

**Trasel, R.** “Usando Padrão de Projeto Builder com Java”, <https://medium.com/@robson.trasel/usando-padr%C3%A3o-de-projeto-builder-com-java-6b7001310e6d>, Accessed October 2025.

**ORACLE.** Java Platform, Standard Edition Documentation. Disponível em: <https://docs.oracle.com/en/java/>. Acesso em: 28 ago. 2025.